高级语言程序设计实验报告

—— 幸存者大作战游戏开发

学校： 南开大学

姓名： 李云鹤

学号： 2512995

班级： 人工智能专业

日期： 2026年4月2日

目录

1 项目基本信息 **2**

2 项目设计思路 **2**

2.1 游戏模式设计 2

2.2 核心系统设计 2

2.2.1 玩家系统 2

2.2.2 敌人系统 2

2.2.3 升级系统 2

2.2.4 UI系统 3

2.3 技术架构设计 3

3 项目实现细节 **3**

3.1 游戏主循环 3

3.2 玩家控制 3

3.3 敌人 3

3.4 碰撞检测 3

3.5 技能系统 4

3.6 升级系统 4

3.7 数据面板 4

4 遇到的问题与解决方案 **4**

5 收获与总结 **4**

6 AI工具使用情况披露 **5**

6.1 代码开发与Bug修复 5

6.2 架构设计与优化建议 5

6.3 调试与问题分析 6

6.4 命名与文案优化 6

1 项目基本信息

项目名称: 幸存者大作战项目类型: 像素平台游戏开发语言: C++ 17 开发框架: Qt 6.9.1 开发环境: Windows 11, Qt Creator, MinGW GCC 项目成员: 个人项目

2 项目设计思路

2.1 游戏模式设计

本项目设计了两种游戏模式：

单人模式：采用"幸存者"类游戏玩法，玩家控制角色在地图上移动，击败不断生成的敌人，获取经验并升级。升级时可以选择强化能力，如增加攻击力、提升速度、减少技能冷却等。游戏目标是在敌人的围攻下生存尽可能长的时间。

多人模式：支持2-4人对抗，玩家之间可以相互攻击，最后存活的玩家获胜。多人模式下玩家拥有不同的控制方式，增加了游戏的趣味性和竞争性。

2.2 核心系统设计

2.2.1 玩家系统

- 实现了多个可选择的角色，每个角色有不同的外观 - 单人模式下实现了技能系统，包括导弹雨、能量护盾、闪电链三种技能 - 多人模式下实现了基本的移动、攻击、跳跃等操作

2.2.2 敌人系统

- 单人模式下定期生成敌人，敌人会自动向玩家移动 - 实现了普通敌人和BOSS，BOSS每3级出现一次，拥有更高的生命值和伤害 - 敌人死亡时会掉落经验球，被玩家拾取后增加经验值

2.2.3 升级系统

- 玩家获得足够经验后升级，显示升级面板 - 每次升级提供三个随机的升级选项，包括攻击力、速度、生命值、攻击速度、技能伤害、技能冷却等 - 升级选项在进入升级页面时随机生成，而不是在选择时变化

2.2.4 UI系统

- 实现了主菜单、玩家数量选择、角色选择、游戏界面、升级面板、数据面板、结算面板等UI界面 - 界面设计简洁明了，操作直观

2.3 技术架构设计

- **核心类**：CGame类作为游戏的主类，负责游戏的初始化、更新、绘制等核心功能 - **游戏循环**：使用QTimer实现游戏主循环，以固定时间步长更新游戏逻辑 - **事件处理**：使用Qt的事件系统处理键盘输入、鼠标点击等事件 - **绘制系统**：使用QPainter在paintEvent中绘制游戏场景 - **资源管理**：使用Qt的资源系统管理图片、音效等资源

3 项目实现细节

3.1 游戏主循环

游戏主循环使用QTimer实现，以16ms为间隔（约60FPS）触发游戏逻辑更新和界面重绘。为了确保游戏在不同性能的设备上运行速度一致，实现了固定时间步长的游戏更新机制，使用QElapsedTimer和accumulator来控制游戏速度。

3.2 玩家控制

单人模式：

- 使用WASD键控制移动 - 空格键攻击 - F键释放技能 - P键打开数据面板

多人模式：

- 玩家1：使用WASD键控制 - 玩家2：使用方向键控制 - 玩家3：使用JKLI键控制 - 玩家4：使用1352键控制

3.3 敌人

敌人使用简单的追逐算法，向玩家位置移动。在单人模式下，敌人会定期生成，数量随玩家等级增加而增加。BOSS每3级出现一次，拥有更高的生命值和伤害。

3.4 碰撞检测

实现了多种碰撞检测：

- 玩家与敌人的碰撞检测 - 子弹与敌人的碰撞检测 - 玩家与经验球的碰撞检测 - 玩家与道具的碰撞检测 - 玩家与地形的碰撞检测（多人模式）

3.5 技能系统

实现了三种技能：

- **导弹雨**：发射10枚追踪导弹，对敌人造成大量伤害 - **能量护盾**：恢复玩家50点生命值 - **闪电链**：攻击多个敌人，最多链3个敌人，并使敌人眩晕1秒

3.6 升级系统

每次升级时，游戏会随机生成三个升级选项，玩家可以选择其中一个进行强化。升级选项包括：

- 攻击力 +5 - 速度 +1 - 生命值 +20 - 攻击速度 +5- 技能伤害 +50 - 技能冷却 -10

3.7 数据面板

按P键可以打开数据面板，显示玩家的详细属性：

- 攻击力 - 攻击速度 - 当前技能 - 技能伤害 - 技能冷却 - 生命值 - 速度 - 等级 - 经验 - 击杀数

4 遇到的问题与解决方案

1. **问题**：多人模式跳跃一次落地后，不能再起跳了 **解决方案**：更新checkCollisions函数，在玩家落地时重置跳跃状态和跳跃次数

2. **问题**：追踪子弹指针访问违规 **解决方案**：在checkBulletCollisions函数中添加try-catch块和额外的指针有效性检查，确保指针安全

3. **问题**：按右方向键时游戏帧率加快 **解决方案**：实现了固定时间步长的游戏更新机制，使用QElapsedTimer和accumulator来控制游戏速度

4. **问题**：单人模式的怪物出现在多人模式中 **解决方案**：修改updateGame函数，只在单人模式下生成和更新敌人；修改checkBulletCollisions函数，只在单人模式下检测敌人碰撞；修改drawGame函数，只在单人模式下渲染敌人

5. **问题**：多人模式下生命值不会改变 **解决方案**：在checkBulletCollisions函数中添加多人模式下的敌人与玩家碰撞检测逻辑

6. **问题**：多人模式下攻击冷却时间过短 **解决方案**：为多人模式下的所有玩家设置了0.8秒的攻击冷却时间

5 收获与总结

通过本次大作业，我获得了以下收获：

1. **深入理解了Qt框架**：对Qt的核心机制，如事件循环、绘图系统、资源管理、事件处理等有了更深入的理解和实践经验

2. **掌握了C++游戏开发基础**：学习了如何运用面向对象思想设计游戏实体类，管理游戏状态，实现基本的游戏逻辑（移动、碰撞、AI、升级系统）

3. **熟悉了游戏开发流程**：从游戏设计、资源准备、代码实现到测试调试，完整地体验了游戏开发的全过程

4. **提升了调试能力**：在解决各种问题时，学会了通过日志分析、断点调试、隔离问题等方法定位和修复Bug，提升了调试复杂程序的能力

5. **理解了游戏物理和逻辑**：实现了固定时间步长的游戏更新机制，确保游戏在不同设备上运行速度一致

6. **认识到代码组织的重要性**：合理的代码结构和命名规范对于大型项目的可维护性至关重要

7. **实践了UI与逻辑分离**：通过将游戏逻辑和UI绘制分离，提高了代码的可读性和可维护性

总结：本项目成功实现了一个具有单人模式和多人模式的像素平台游戏，涵盖了图形界面、事件处理、游戏逻辑、资源管理等多个方面。开发过程充满挑战，但也极大地锻炼了我的C++编程能力、Qt应用能力和问题解决能力。虽然仍有一些待完善之处（如更丰富的敌人类型、更复杂的升级系统、性能优化等），但基本达到了预期的目标。

6 AI工具使用情况披露

AI工具名称及版本：Claude 3.5 Sonnet

使用场景与具体贡献：

6.1 代码开发与Bug修复

- **使用方式**：在开发过程中遇到技术难题时，向AI咨询解决方案 - **具体解决的问题**：

- **代码生成比例**：约15%的核心逻辑代码参考了AI提供的方案，所有代码均经过人工理解和修改

6.2 架构设计与优化建议

- **使用方式**：咨询游戏架构设计和代码组织方面的最佳实践 - **具体建议**：

- 游戏循环的设计模式选择

问题描述	AI提供的解决方案	实际应用效果
追踪子弹指针访问违规	添加try-catch块和指针有效性检查	成功修复崩溃问题
游戏帧率不稳定	实现固定时间步长的游戏更新机制	游戏运行更加流畅稳定
多人模式角色死亡不消失	在绘制函数中添加生命值检查	角色死亡后正确消失
数据面板显示格式化错误	修正QString的arg()方法使用方式	数据正确显示为数字
角色跳跃后无法再次起跳	在碰撞检测中重置跳跃状态	跳跃功能恢复正常

- 碰撞检测算法的优化方向
- 资源管理（图片、音效）的加载策略
- UI与游戏逻辑分离的实现方式

- **采纳情况**：约60%的建议被采纳并应用到项目中

6.3 调试与问题分析

- **使用方式**：将错误日志和代码片段提供给AI进行分析 - **分析案例**：

- 分析追踪子弹目标丢失异常的原因
- 诊断多人模式下生命值不减少的问题
- 排查敌人与玩家碰撞检测失效的原因

- **解决效率**：使用AI辅助后，平均问题解决时间缩短约40%

6.4 命名与文案优化

- **使用方式**：咨询游戏名称、界面文案的优化建议 - **具体应用**：

- 游戏名称从"像素平台游戏"优化为"幸存者大作战"
- 界面提示文案的优化（如游戏模式说明、操作提示等）
- 数据面板中属性名称的规范化

使用统计：

- 总交互次数：约50次对话 - 代码相关咨询：约30次 - 文档撰写辅助：约10次 - 问题调试辅助：约10次

声明：

本项目的核心游戏逻辑、玩法设计、资源制作（图片、音效）均由开发者独立完成。AI工具主要用于辅助解决技术难题、优化代码质量和提升文档撰写效率。所有AI生成的内容都经过人工审核、理解和修改，确保符合项目实际需求。